

CS635 Assignment 2

- Palti Ramyasri
- 190050078

1.2 Preprocessing

– bow.py

- Run the command: `python bow.py --dir {dir_path} --bbow {bbow_destindation_file} --cbow {cbow_destination_file}`
 - {dir_path} is path of directory relative to root directory for which bbow.json and cbow.json have to be computed and saved
 - {bbow_destination_file} is path of bbow.json that has to be saved relative to the root directory
 - {cbow_destination_file} is path of cbow.json that has to be saved relative to the root directory
- Implementation
 - Load 'en_core_web_sm' to tokenise documents
 - For each file in the directory(recursively)
 - Tokenize the document
 - For each token, assign 1 in dictionary for bbow computation
 - For each token, increment count in dictionary for cbow computation
- get_dict_file performs a cbow/bbow dictionary for a document
 - Arguments
 - bbow_flag - 0 for cbow, and 1 fo bbow
 - dir_name - directory in which the document is
 - file_name - name of the document
- get_all_files_dict performs cbow/bbow dictionary for all documents in directory
 - Arguments
 - bbow_flag - 0 for cbow, and 1 fo bbow
 - parent_dir - parent directory in which all category directories are present
 - dir_list - list of directories that have documents

1.3 Naive Bayes Multinomial

– helper.py

- Saves vocabulary as vocab.npy and P(y) as probab_y.json

– multinom_train.py

- Run the command: `python multinom_train.py --type {bbow/cbow} --laplace {laplace_constant}`
- Implementation
 - Train the classifier with $P(v|y)$ values, v is a word in vocabulary and y is the label of document
 - For a word v and document d , which has label y
 - $P(v|y) = (\sum_{\text{docs } d' \text{ of } D \text{ of class } y} f[d', v] + k) / (\sum_{w \text{ of } V} \sum_{\text{docs } d' \text{ of } D \text{ of class } y} f[d', w] + |V|k)$
 - k - Laplace constant
 - V - vocabulary
 - $f[d', v]$ - frequency of word v in document d' (0/1 for bbow, and count for cbow)
 - Assign a value to k , and perform training
 - We tune k according to the accuracy we get with testing data
 - Perform numerator computation for each word in vocabulary for all labels
 - Get the denominator by summing up all the numerators for a given label
 - If a word v has $f[d', v]$ as zero, set the numerator to the smoothing constant $1/|V|$ ($|V|$ is the length of vocabulary) - for smoothing
 - Save logarithmic value of $P(v|y)$ in json file in the format, $\{y1' : \{v1' : , v2' : , \dots\}, y2' : \{...\}\}$ – saved as `p_v_y_cbow.json` and `p_v_y.bbow.json`

–`multinom_test.py`

Before predicting labels for documents, compute `bbow.json` and `cbow.json` for the directory of test files -done to decrease testing time

- `get_prediction` → predicts label for a test document
 - $y'_d = \operatorname{argmax}_y P(d|y).P(y)$ → perform log scale to get prediction → so sum all probabilities while computing argmax label
 - $P(d|y) = \prod_{v \text{ of } d} P(v|y)$
 - For a document d ,
 - For each label y ,
 - For each word v in doc d , get $P(v|y)$ and add it to probability sum
 - Find label which has maximum log probability sum, assign that label to doc d
 - If word seen is not in vocabulary, add $\text{np.log}(\text{smoothing constant})$ for smoothing, smoothing constant = $1/(\text{length of vocab})$
- `Get_all_predictions` → predicts labels for all test documents and computes accuracy

1.3 Naive Bayes Poisson

For C-Bow

– `poisson_train.py`

- Run the command: `python poisson_train.py --alpha {alpha value} --beta {beta value}`
- Implementation
 - Train the classifier with lambda values for a word v and label y , v is a word in vocabulary and y is the label of document

- For a word v and document d , which has label y
 - $\lambda(v|y) = (\sum_{\text{docs } d' \text{ of class } y} f[d', v] + \alpha) / (|\text{files with label } y| + \beta)$
 - Alpha and beta are hyperparameters
 - $f[d', v]$ - frequency of word v in document d' (0/1 for bbow, and count for cbow)
 - Assign values to alpha and beta, and perform training
 - We tune alpha and beta, according to the accuracy we get with testing data
 - Get summation of $f[d', v]$ with cbow dictionary stored
 - Save logarithmic value of $\lambda(v|y)$ in json file in the format, $\{ 'y1' : \{ 'v1' : , 'v2' : , \dots \}, 'y2' : \{ \dots \} \}$ – saved as `lambda_v_y.json`

–poisson_test.py

Before predicting labels for documents, compute cbow.json for the directory of test files -done to decrease testing time

- `get_prediction` → predicts label for a test document
 - $y'_d = \operatorname{argmax}_y P(d|y).P(y)$ → perform log scale to get prediction → so sum all probabilities while computing argmax label
 - $P(d|y) = \prod_{v \text{ of } d} P(v|y)$
 - $P(v|y) = e^{-\lambda} * \lambda^k / k!$
 - For a document d ,
 - For each label y ,
 - For each word v in doc d , get $P(v|y)$ and add it to probability sum
 - Find label which has maximum log probability sum, assign that label to doc d
 - If word seen is not in the vocabulary, lambda would be $\alpha / (|\text{number of files with label } y| + \beta)$ for smoothing
 - `Get_all_predictions` → predicts labels for all test documents and computes accuracy

Results

- Hyperparameter tuning
 - Multinomial Classifier accuracy
 - 0.82129580456718 -- k 0.5 cbow
 - 0.8171800318640468 -- k 0.5 bbow
 - 0.7441582580987786 -- k 2 cbow
 - 0.8146574614976102 – k 10^{-6} bbow
 - 0.8235528412108338 – k 10^{-6} cbow
 - Poisson Classifier Accuracy
 - alpha 2 beta 0.5 → 0.6958311205523101
 - alpha 2 beta 1 → 0.6958311205523101
 - alpha 1 beta 0.25 → 0.7234466277217206

- $\alpha 0.5 \beta 0.25 \rightarrow 0.7399097185342538$
- Chosen hyperparameter values -
 - Multinomial : k 0.5
 - C-Bow accuracy - 0.8213
 - B-Bow accuracy - 0.8172
 - Poisson - $\alpha 0.5 \beta 0.25$
 - C-Bow accuracy - 0.7399